# Programs on C++

**//Q1) Print addresses of variables.**

```
#include <iostream>
using namespace std;
int main() {
   int age = 30;
   float sal = 1500.50f;
   cout << "Address of age = " << &age << endl;
   cout << "Address of salary = " << &sal << endl;
   return 0;
}
//Output  Address of age = 0x7ffe8f99bbac
Address of salary = 0x7ffe8f99bba8
```

**//Q2) Demonstrate pointer basics.**

```
#include <iostream>
using namespace std;
int main() {
   int a = 87;
   int *p1 = &a;
   cout << "Address of a = " << &a << endl;
   cout << "Address of p1 = " << &p1 << endl;
   cout << "Value of p1 = " << p1 << endl;
   cout << "Value of a = " << a << endl;
   cout << "Value of a = " << *p1 << endl;
   cout << "Value of a = " << *(&a) << endl;
   return 0;
}
//Output
```

Address of a = 0x7ffc6833de6c

Address of p1 = 0x7ffc6833de60

Value of p1 = 0x7ffc6833de6c

Value of a = 87

Value of a = 87

Value of a = 87

**Q3) double pointer usage.**

```
#include <iostream>
using namespace std;
int main()
{
   int a = 87;
   int *p1 = &a;
   int **q1 = &p1;
   cout << "Address of a = " << &a << endl;
   cout << "Address of p1 = " << &p1 << endl;
   cout << "Address of q1 = " << &q1 << endl;
   cout << "Value of p1 = " << p1 << endl;
   cout << "Value of q1 = " << q1 << endl;
   cout << "Value of a = " << a << endl;
   cout << "Value of a = " << *p1 << endl;
   cout << "Value of a = " << *(&a) << endl;
   cout << "Value of a = " << **q1 << endl;
   return 0;
}
```

**//Q4) Find size of pointers and their values.**

```
#include <iostream>
using namespace std;
int main()
```

```
{
   char a = 'x', *p1 = &a;
   int b = 12, *p2 = &b;
   float c = 2.5f, *p3 = &c;
   double d = 18.33, *p4 = &d;
   cout << "sizeof(p1) = " << sizeof(p1)
        << ", sizeof(*p1) = " << sizeof(*p1) << endl;
   cout << "sizeof(p2) = " << sizeof(p2)
        << ", sizeof(*p2) = " << sizeof(*p2) << endl;
   cout << "sizeof(p3) = " << sizeof(p3)
        << ", sizeof(*p3) = " << sizeof(*p3) << endl;
   cout << "sizeof(p4) = " << sizeof(p4)
        << ", sizeof(*p4) = " << sizeof(*p4) << endl;
   return 0;
}
//Output:
sizeof(p1) = 8, sizeof(*p1) = 1
sizeof(p2) = 8, sizeof(*p2) = 4
sizeof(p3) = 8, sizeof(*p3) = 4
sizeof(p4) = 8, sizeof(*p4) = 8
```

**//Q5) Show pointer arithmetic for different data types.**

```
#include <iostream>
using namespace std;
int main()
{
   int a = 5, *pi = &a;
   char b = 'x', *pc = &b;
   float c = 5.5f, *pf = &c;
   cout << "Value of pi (Address of a) = " << pi << endl;
```

```
    cout << "Value of pc (Address of b) = " << static_cast<void*>(pc) << endl;
    cout << "Value of pf (Address of c) = " << pf << endl;
    pi++;
    pc++;
    pf++;
    cout << "Now value of pi = " << pi << endl;
    cout << "Now value of pc = " << static_cast<void*>(pc) << endl;
    cout << "Now value of pf = " << pf << endl;
    return 0;
}

//Output:
Value of pi (Address of a) = 0x7ffcfd33c8a4
Value of pc (Address of b) = 0x7ffcfd33c8a3
Value of pf (Address of c) = 0x7ffcfd33c89c
Now value of pi = 0x7ffcfd33c8a8
Now value of pc = 0x7ffcfd33c8a4
Now value of pf = 0x7ffcfd33c8a0
```

**//Q6) Access array using pointers.**

```
#include <iostream>
using namespace std;
int main()
{
    int arr[5] = {5, 10, 20, 25, 30};
    for (int i = 0; i < 5; i++)
    {
        cout << "Value of arr[" << i << "]" << endl;
        cout << arr[i] << endl;
        cout << i[arr] << endl;
        cout << *(arr + i) << endl;
```

```
        cout << *(i + arr) << endl;
    }
    return 0;
}
```

**//Q7) Type casting using pointer.**

```
#include <iostream>
using namespace std;
int main()
{
    int a = 66;
    char *c;
    c = (char*)&a;
    cout << (int)*c << endl;
    cout << *c << endl;
    return 0;
}
```

O/p

66

B

**//Q8) Traverse array using pointer in function.**

```
#include <iostream>
using namespace std;
void fun(int *p);
int main()
{
    int arr[] = {3, 6, 9, 12, 15, 18};
    fun(arr);
    return 0;
}
```

```
void fun(int *p)
{
   for (int i = 0; i < 6; i++)
   {
      cout << *p << endl;
      p++;
   }
}
//Output:
3
6
9
12
15
18
```

**//Q9) null pointer.**

```
#include <iostream>
using namespace std;
int main()
{
   int *ptr = nullptr;
   cout << "The value of ptr is " << ptr << endl;
   return 0;
}
```

```
//Output:
The Value of ptr is (nil)
```

**//Q10) Count frequency of element in array.**

```
#include <iostream>
```

```
using namespace std;
int count(int arr[], int size, int target)
{
   int count = 0;
   for(int i = 0; i < size; i++)
   {
      if(arr[i] == target)
      {
         count++;
      }
   }
   if(count > 0)
      return count;
   else
      return -1;
}
int main()
{
   int arr[] = {3,4,6,7,2,3,2,4,2};
   int size = sizeof(arr) / sizeof(arr[0]);
   int target = 2;
   int result = count(arr, size, target);
   if(result != -1)
   {
      cout << "Count = " << result << endl;
   }
   else
   {
      cout << "-1" << endl;
   }
   return 0;
```

```
}
//Output
 Count = 3
```

**//Q11) Find last occurrence of element.**

```
#include <iostream>
using namespace std;
int last(int arr[], int size, int target)
{
   int index=-1;
   for(int i = 0; i < size; i++)
   {
      if(arr[i] == target)
      {
        index=i;
      }
   }
   return index;
}
int main()
{
   int arr[] = {3,4,6,7,2,3,2,4,2,9};
   int size = sizeof(arr) / sizeof(arr[0]);
   int target = 2;
   int result = last(arr, size, target);
cout<<"last index"<<result;
   return 0;
}

//Output last index8
```

**//Q12) Check array palindrome.**

```
#include <iostream>
using namespace std;
bool isPalindrome(int arr[], int size)
{
   int start = 0;
   int end = size - 1;
   while(start<end)
   {
      if(arr[start] != arr[end])
      {
         return false;
      }
      start++;
      end--;
   }
   return true;
}
int main()
{
   int arr[] = {1, 2, 3, 2, 1};
   int size = sizeof(arr) / sizeof(arr[0]);
   if(isPalindrome(arr, size))
   {
      cout << "Palindrome";
   }
   else
   {
```

```
        cout << "Not a Palindrome";
    }
    return 0;
}
```

**//Q13) Find common elements in arrays.**

```
#include <iostream>
using namespace std;
int main()
{
    int arr1[] = {1, 2, 3, 4, 5};
    int arr2[] = {3, 5, 7, 8, 9};
    int size1 = sizeof(arr1) / sizeof(arr1[0]);
    int size2 = sizeof(arr2) / sizeof(arr2[0]);
    cout << "Common elements are: ";
    for(int i = 0; i < size1; i++)
    {
        for(int j = 0; j < size2; j++)
        {
            if(arr1[i] == arr2[j])
            {
                cout << arr1[i] << " ";
            }
        }
    }
    return 0;
}
```

**//Q14) Sort 0s, 1s, 2s array.**

```
#include <iostream>
```

```
using namespace std;
int main() {
   int n;
   cin >> n;
   int arr[n];
   for(int i = 0; i < n; i++) {
      cin >> arr[i];
   }
   int low = 0, mid = 0, high = n - 1;
   while(mid <= high) {
      if(arr[mid] == 0) {
         swap(arr[low], arr[mid]);
         low++;
         mid++;
      }
      else if(arr[mid] == 1) {
         mid++;
      }
      else {
         swap(arr[mid], arr[high]);
         high--;
      }
   }
   for(int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }
   return 0;
}
```

**//Q15) Print prime numbers up to N.**

```
#include <iostream>
using namespace std;
bool isPrime(int n)
{
   if (n < 2)
   {
      return false;
   }
   for (int i=2; i*i<=n;i++)
   {
      if (n%i==0)
         return false;
   }
   return true;
}
int main()
{
   int n;
   cin >> n;
   for (int i=2;i<= n; i++)
   {
      if (isPrime(i))
      {
         cout << i << " ";
      }
   }
   return 0;
}
```

**//Q16) Sum of array elements.**

```
#include <iostream>
```

```
using namespace std;
int calculate(int arr[], int size)
{
   int sum = 0;

   for(int i = 0; i < size; i++)
   {
      sum = sum + arr[i];
 }
   return sum;
}
int main()
{
   int arr[] = {10, 20, 30, 40, 50};
   int size = sizeof(arr) / sizeof(arr[0]);
   cout << "Sum = " << calculate(arr, size);
   return 0;
}
```

**//Q17) Merge two sorted linked lists.**

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node* next;
};
```

```
Node* mergeLists(Node* head1, Node* head2)
{
   if(head1 == NULL)
      return head2;

   if(head2 == NULL)
      return head1;

   Node* result = NULL;

   if(head1->data <= head2->data)
   {
      result = head1;
      result->next = mergeLists(head1->next, head2);
   }
   else
   {
      result = head2;
      result->next = mergeLists(head1, head2->next);
   }

   return result;
}

void printList(Node* head)
{
   while(head != NULL)
   {
      cout << head->data << " ";
      head = head->next;
   }
```

```
}

int main()
{
   Node* head1 = new Node{1, new Node{3, new Node{5, NULL}}};
   Node* head2 = new Node{2, new Node{4, new Node{6, NULL}}};

   Node* merged = mergeLists(head1, head2);

   printList(merged);

   return 0;
}
```

**//Q18) Print 1 to N recursively.**

```
#include <iostream>
using namespace std;
void printnumbers(int n)
{
   if(n==0)
   {
      return ;
   }
   printnumbers(n-1);
   cout<<n<<" ";
}
int main()
{
   int n=5;
   printnumbers(n);
   return 0;
```

```
}
//Output
1 2 3 4 5
```

**//Q19) Print N to 1 recursively.**

```
#include <iostream>
using namespace std;
void printReverse(int n)
{
   if(n==0)
   {
      return ;
   }
    cout<<n<<" ";
   printReverse(n-1);
}
int main()
{
   int n=5;
   printReverse(5);
   return 0;
}
//Output
5 4 3 2 1
```

**//Q20) Factorial using recursion.**

```
#include <iostream>
using namespace std;
int factorial(int n)
```

```
{
   if(n==0 || n==1)
   {
      return 1;
   }
 return n * factorial(n-1);
}
int main()
{
   int n=5;
   cout<<factorial(5);
   return 0;
}
//Output : 120
```

**//Q21)Power of number**

```
#include <iostream>
using namespace std;
int power(int base,int exp)
{
   if(exp==0)
   {
      return 1;
   }
 return base * power(base,exp-1);
}
int main()
{
   int n=5;
   cout<<power(2,5);
```

```
    return 0;
}
//Output 32
```

**//Q22) Reverse string recursively.**

```
#include <iostream>
using namespace std;
void reverseString(char str[], int start, int end)
{
    if(start >= end)
        return;
    char temp = str[start];
    str[start] = str[end];
    str[end] = temp;
    reverseString(str, start + 1, end - 1);
}
int main()
{
    char str[] = "HELLO";
    reverseString(str, 0, 4);
    cout << str;
    return 0;
}
```

**//Q23) Remove spaces from string.**

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
```

```
    string str;
    cout << "Enter a string: ";
    cin>> str;
    string result = "";

    for (int i = 0; i < str.length(); i++)
    {
        if (str[i] != ' ')
        {
            result += str[i];
        }
    }
    cout << "String after removing spaces: " << result;
    return 0;
}

//Output
Enter a string: hello world
String after removing spaces: helloworld
```

**//Q24) First non-repeating element.**

```
#include <iostream>
using namespace std;
int firstNonRepeating(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        int count = 0;
        for (int j = 0; j < n; j++)
        {
            if (arr[i] == arr[j])
```

```
            count++;
        }
        if (count == 1)
            return arr[i];
    }
    return -1;
}
int main()
{
    int arr[] = {4, 5, 1, 2, 1, 2, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = firstNonRepeating(arr, n);
    if (result != -1)
        {
        cout << "First non-repeating element: " << result;
        }
    else
    {
        cout << "No non-repeating element found";
}
    return 0;
}
//Output
First non-repeating element: 5
```

**//Q25) Split array into chunks.**

```
#include <iostream>
using namespace std;
void splitIntoChunks(int arr[], int n, int chunkSize)
{
    for (int i = 0; i < n; i=i+ chunkSize)
```

```
    {
        cout << "[ ";
        for (int j = i; j < i + chunkSize && j < n; j++)
        {
            cout << arr[j] << " ";
        }
        cout << "]" << endl;
    }
}
int main()
{
    int arr[] = {1,2,3,4,5,6,7,8};
    int n = sizeof(arr) / sizeof(arr[0]);
    int chunkSize = 3;
    splitIntoChunks(arr, n, chunkSize);
    return 0;
}

//Output
[ 1 2 3 ]
[ 4 5 6 ]
[ 7 8 ]
```

**//Q26) Sort array using STL sort.**

```
#include <iostream>
#include<algorithm>
using namespace std;
int main()
{
    int arr[]={3,5,1,2,4};
//sort array in ascending order
```

```
    sort(begin(arr),end(arr));
    for( int i:arr)
    {
        cout<<i<<" ";
    }
    return 0;
}
//Output
1 2 3 4 5
```

**//Q27) Sort and display array before/after.**

```
#include <iostream>
#include <algorithm>
using namespace std;
void show(int a[],int array_size)
{
    for(int i=0;i<array_size;i++)
    {
        cout<<a[i]<<" ";
    }
}
int main()
{
    int arr[]={1,5,8,9,6,7,3,4,2,0};
    int asize=sizeof(arr)/sizeof(arr[0]);
    cout<<"The array before sorting is:\n";
    show(arr,asize);
    sort(arr,arr+asize);
    cout<<"\n\nThe array after sorting is:\n";
    show(arr,asize);
    return 0;
```

```
}
```

```
//Output
The array before sorting is:
1 5 8 9 6 7 3 4 2 0
The array after sorting is:
0 1 2 3 4 5 6 7 8 9
```

**//Q28) Sort and binary search.**

```
#include <iostream>
#include <algorithm>
using namespace std;
void show(int a[],int arraysize)
{
   for(int i=0;i<arraysize;++i)
   {
      cout<<a[i]<<" , ";
   }
}
int main()
{
   int a[]={1,5,8,9,6,7,3,4,2,0};
   int asize=sizeof(a)/sizeof(a[0]);
   cout<<"The array before sorting is:\n";
   show(a,asize);
   sort(a,a+asize);
   cout<<"\n\nThe array after sorting is:\n";
   show(a,asize);
   if(binary_search(a,a+10,2))
   {
```

```
      cout<<"\nElement found in the array";
   }
   else
   {
      cout<<"\nElement not found";
   }
   return 0;
}
```

```
//Output
The array before sorting is:
1 , 5 , 8 , 9 , 6 , 7 , 3 , 4 , 2 , 0 ,
The array after sorting is:
0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 ,
Element found in the array
```

**//Q29) Stack operations using STL.**

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
  stack<int>stack;
  stack.push(21);
  stack.push(22);
  stack.push(23);
  stack.push(24);
  stack.push(25);
  cout<<stack.top();
  int num=0;
  stack.push(num);
  stack.pop();
```

```
  stack.pop();
  while(!stack.empty())
  {
     cout<<stack.top()<<" ";
     stack.pop();
  }
   return 0;
}
//Output
2524 23 22 21
```

**//Q30) Queue operations using STL.**

```
#include <iostream>
#include <queue>
using namespace std;
void showq(queue<int> gq)
{
   queue<int> g = gq;
   while(!g.empty())
   {
      cout << '\t' << g.front();
      g.pop();
   }
   cout << '\n';
}
int main()
{
   queue<int> que;
   que.push(11);
   que.push(22);
```

```
    que.push(33);
    cout << "The queue que is :";
    showq(que);
    cout << "\n que.size():" << que.size();
    cout << "\n que.front():" << que.front();
    cout << "\n que.back():" << que.back();
    cout << "\n que.pop():";
    que.pop();
    showq(que);
    return 0;
}

//Output
The queue que is :11 22 33
 que.size():3
 que.front():11
 que.back():33
 que.pop():22  33
```

**//Q31) Count spaces and special characters**

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    char str[100];
    int special = 0, whitespace = 0;
    cout << "Enter a message: ";
    cin.getline(str, 100);
    for(int i = 0; str[i] != '\0'; i++)
```

```
    {
        if(str[i] == ' ')
        {
            whitespace++;
        }
        else if(!isalnum(str[i]))
        {
            special++;
        }
    }
    cout << "Number of white spaces: " << whitespace << endl;
    cout << "Number of special characters: " << special << endl;
    return 0;
}
//Output
Enter a message: dsa@#1234d
Number of white spaces: 0
Number of special characters: 2
```

**//Q32) Insert at beginning in linked list.**

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *next;
};
void insertBegin(Node *&head, int value)
{
    Node *newNode = new Node;
```

```
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}
void display(Node *head)
{
    while (head != NULL)
    {
        cout << head->data << " ";
        head = head->next;
    }
}
int main()
{
    Node *head = NULL;
    insertBegin(head, 10);
    insertBegin(head, 20);
    insertBegin(head, 30);
    display(head);
    return 0;
}
//Output
30 20 10
```

**//Q33) Delete element from array.**

```
#include<iostream>
using namespace std;
int main()
{
    int arr[10]={1,2,3,4,5};
```

```
    int n=5;
    int element=3;
    int pos=-1;
    for(int i=0;i<n;i++)
    {
        if(arr[i]==element)
        {
            pos=i;
            break;
        }
    }
    if(pos!=-1)
    {
        for(int i=pos;i<n-1;i++)
        {
            arr[i]=arr[i+1];
        }
        n--;
    }
    cout<<"List: ";
    for(int i=0;i<n;i++)
    {
        cout<<arr[i]<<" ";
    }
    return 0;
}
//Output
List: 1 2 4 5
```

**//Q34) Vector operations in STL.**

```
#include<iostream>
```

```
#include<vector>
using namespace std;
int main()
{
   vector<int> g1;
   for (int i=0;i<=5;i++)
   {
      g1.push_back(i);
   }
   cout<<"Output of begin and end"
;
   for(auto i=g1.begin();i!=g1.end();i++)
   {
      cout<<*i<<" ";
   }
   cout<<"\n Output of rbegin and rend :";
   for(auto ir=g1.rbegin();ir != g1.rend();++ir)
   {
      cout<<*ir<<"";
   }
   cout<<"\nSize:"<<g1.size();
   cout<<"\nCapacity:"<<g1.capacity();
   cout<<"\nMax_Size:"<<g1.max_size();
   cout<<"\nat \t: g1.at(4)="<<g1.at(3);
   cout<<"\nfront \t: g1.front(4)="<<g1.front();
   cout<<"\nback \t: g1.back(4)="<<g1.back();
   return 0;
}
//Output
Output of begin and end0 1 2 3 4 5
 Output of rbegin and rend :543210
```

```
Size:6
Capacity:8
Max_Size:2305843009213693951
at        : g1.at(4)=3
front     : g1.front(4)=0
back      : g1.back(4)=5
```

**//Q35) Display number in hex, octal, decimal.**

```
#include<iostream>
using namespace std;
int main()
{
  int a=10;
  cout<<"Hex="<<hex<<a<<endl;
  cout<<"Octal="<<oct<<a<<endl;
  cout<<"Decimal="<<dec<<a<<endl;
   return 0;
}
//Output
Hex=a
Octal=12
Decimal=10
```

**//Q36) Output formatting using setw().**

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
```

```
   int a = 200;
   cout << setw(8) << a << endl;
   cout << setw(5) << a << setw(6) << a << endl;
   cout << setw(3) << a << setw(10) << a << endl;
   return 0;
}
//Output

  200
 200   200
200      200
```

**//Q37) Formatting using setw() and setfill().**

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
   int a = 20;
   cout<<setfill('_');
   cout << setw(8) << a << endl;
   cout << setw(5) << a << setw(6) << a << endl;
   cout << setw(3) << a << setw(10) << a << endl;
   return 0;
}

//Output

______20
___20____20
_20________20
```